

YouTube Video Transcript RAG Indexer

This script automates the process of extracting a transcript from a YouTube video, splitting it into manageable chunks, generating vector embeddings, and storing them in a Qdrant vector database. This is a foundational step for building a Retrieval-Augmented Generation (RAG) system based on YouTube content.

Overview

The workflow of this script consists of four main stages:

1. **Extraction:** Downloading the transcript and metadata from a provided YouTube URL.
2. **Processing:** Splitting the extracted text into smaller, overlapping chunks suitable for embedding models.
3. **Embedding:** Converting the text chunks into numerical vectors using a specific embedding model hosted on OpenRouter.
4. **Storage:** Indexing and saving the vector embeddings along with their text chunks into a local Qdrant database.

Prerequisites & Dependencies

To run this script, you need the following libraries installed:

- `langchain_yt_dlp` : For extracting YouTube transcripts and metadata (often requires `yt-dlp` installed system-wide or via pip).
- `langchain_text_splitters` : For chunking the text.
- `langchain_openai` : For generating embeddings using OpenAI's API format (used here with OpenRouter).
- `langchain_qdrant` : For connecting to and interacting with the Qdrant vector database.
- `python-dotenv` : For loading environment variables securely.
- `qdrant-client` : (Implicit dependency) Required to run Qdrant locally, usually via Docker (`docker run -p 6333:6333 qdrant/qdrant`).

Environment Variables: You must have a `.env` file in the same directory as the script with the following variable defined:

- `OPENROUTER_API_KEY` : Your API key from OpenRouter.ai.

Step-by-Step Code Explanation

1. Initialization and Setup

```
from pathlib import Path
from langchain_yt_dlp.youtube_loader import YoutubeLoaderDL
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_qdrant import QdrantVectorStore
from dotenv import load_dotenv
import os
```

```
load_dotenv()
```

- **Imports:** Brings in necessary classes from LangChain and standard Python libraries.
- `load_dotenv()` : Reads the `.env` file and loads the variables (specifically `OPENROUTER_API_KEY`) into the script's environment, preventing hardcoding sensitive keys.

2. Loading the YouTube Transcript

```
video_url = "https://www.youtube.com/watch?v=XXXXXXXXXX" # replace with your

# Load transcript + metadata (title, author, description, view_count, publish_d
loader = YoutubeLoaderDL.from_youtube_url(
    video_url,
    add_video_info=True,
)
docs = loader.load()
print(docs[0])
```

- `video_url` : The target video to process. **Note:** This needs to be replaced with a valid YouTube URL before execution.
- `YoutubeLoaderDL` : Uses `yt-dlp` under the hood to fetch the video's transcript.
- `add_video_info=True` : Ensures that metadata (like author, publish date, title) is included in the resulting `Document` object's metadata dictionary.
- `loader.load()` : Executes the extraction, returning a list of LangChain `Document` objects. Typically, for a single video, this returns a list with one item (`docs[0]`).

3. Text Chunking

```
# Split docs into smaller chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=400
)

chunks = text_splitter.split_documents(documents=docs)
```

- **Purpose:** Large transcripts exceed the context window limits of embedding models. Chunking breaks them down.
- `RecursiveCharacterTextSplitter` : A smart splitter that tries to split on paragraph breaks, then sentences, then words, keeping related text together.
- `chunk_size=1000` : Each resulting chunk will be approximately 1000 characters long.
- `chunk_overlap=400` : To preserve context between chunks, the last 400 characters of chunk *N* will overlap with the first 400 characters of chunk *N+1*.
- `chunks` : A new list containing the multiple, smaller `Document` objects.

4. Configuring the Embedding Model

```
# Vector Embeddings via OpenRouter (OpenAI-compatible /v1/embeddings)
embedding_model = OpenAIEmbeddings(
    model='nvidia/llama-nemotron-embed-v1-1b-v2:free',
    api_key=os.environ['OPENROUTER_API_KEY'],
    base_url='https://openrouter.ai/api/v1',
    check_embedding_ctx_length=False,
    model_kwargs={"encoding_format": "float"},
)
```

- **Purpose:** Converts the text chunks into numerical vectors (embeddings) so they can be compared mathematically for similarity searches later.
- `OpenAIEmbeddings` : Even though we aren't using OpenAI, OpenRouter provides an OpenAI-compatible API endpoint, allowing us to use this standard LangChain wrapper.
- `model` : Specifies the specific free model hosted on OpenRouter (`nvidia/llama-nemotron-embed-v1-1b-v2:free`).
- `base_url` : Redirects the API call from OpenAI's default servers to OpenRouter's servers.

5. Vector Database Storage

```
vector_store = QdrantVectorStore.from_documents(
    documents=chunks,
    embedding=embedding_model,
    url='http://localhost:6333',
    collection_name='learning_rag'
)

print('Indexing of video transcript done.....')
```

- **Purpose:** Stores the generated vectors and the original text chunks in a database optimized for similarity search.
- `QdrantVectorStore.from_documents()` : This is a convenience method that takes the chunks, automatically passes them through the `embedding_model` , and stores the results.
- `url='http://localhost:6333'` : Points to a local instance of Qdrant (which must be running before executing this script).
- `collection_name='learning_rag'` : The name of the "table" or index within Qdrant where these specific vectors will be saved. If the collection doesn't exist, Qdrant will create it.

Usage Notes

1. **Start Qdrant:** Ensure your local Qdrant instance is running on port 6333 before executing the script.
2. **Update URL:** Don't forget to change the `video_url` variable to your desired video.
3. **API Limits:** Be mindful of rate limits on the free OpenRouter model if you are processing very long videos or multiple videos in a loop.